

INTERNSHIP PROJECT REPORT

ON

BANK MANAGEMENT SYSTEM

Submitted as a part of the requirements for the completion of the
Full Stack Web Development Internship Program

Submitted By

SUSHMITA PODDAR

B. Tech – Computer Science Engineering (AI & ML)

Session: 2025–26

Upskill Campus

in association with

UniConverge Technologies Pvt. Ltd.

2026

1.PREFACE

This report presents the work completed during my Full Stack Web Development Internship Program. The internship provided an opportunity to apply web development concepts to a practical project and gain hands-on experience in designing and developing a web-based application.

During the internship, I developed a Bank Management System using full-stack web development technologies. The project is designed to perform basic banking operations such as user registration, login, account management, balance checking, deposit, withdrawal, money transfer and transaction history management through a web-based interface.

The internship helped me strengthen my understanding of frontend development, backend development, database management and client-server communication. I gained practical experience in technologies such as HTML, CSS, JavaScript, backend programming, database connectivity and web APIs.

The internship also provided experience in debugging, testing, handling user input, validating data and managing project files using Git and GitHub.

The internship followed a structured approach that included understanding the assigned problem, planning the solution, designing the application, implementing frontend and backend functionality, connecting the database, testing the application and documenting the final work.

Overall, this internship provided valuable practical exposure to full-stack web development and helped me improve my programming, problem-solving, database management and software development skills.

2. INTRODUCTION

This section provides an overview of the organizations associated with the internship program and the objectives of the internship. The internship was undertaken as a practical learning experience to develop technical skills and gain exposure to project-based full-stack web development.

2.1 ABOUT UNICONVERGE TECHNOLOGIES PVT. LTD.

UniConverge Technologies Pvt. Ltd. (UCT) is a technology company working across areas including Internet of Things (IoT), Wireless Communication, Industry 4.0 and Predictive Maintenance. The organization works with emerging technologies such as Cloud Computing, Machine Learning, 4G/5G, LoRaWAN, Python and other software and communication technologies.

Through its internship and industrial programs, UCT provides students with opportunities to gain practical exposure to current technologies and work on project-based assignments.

The organization encourages learners to understand practical applications of technology and develop technical skills relevant to software and technology industries.

2.2 ABOUT UPSKILL CAMPUS

upSkill Campus is a learning platform that provides students and professionals with courses, internships and practical learning opportunities in various technology domains.

The platform offers internship programs in areas such as Full Stack Web Development, Python, Data Science and Machine Learning, Cloud Computing, Cyber Security, 5G, Embedded Systems and IoT.

The internship program provides students with an opportunity to develop practical skills, work on projects and improve their understanding of technologies relevant to their career goals.

2.3 THE IoT ACADEMY

The IoT Academy is an organization focused on technology education and practical learning in emerging technology domains. Its programs cover areas such as Internet of Things, Embedded Systems, Cloud Computing, Artificial Intelligence and Machine Learning.

The organization aims to provide learners with structured technical education, practical exposure and opportunities to develop industry-relevant skills.

2.4 OBJECTIVES OF THE INTERNSHIP PROGRAM

The main objectives of the internship program were:

- To gain practical experience in full-stack web development.
- To apply web development concepts to a practical project.
- To improve understanding of frontend and backend development.
- To develop problem-solving and debugging skills.
- To understand database management and data storage.
- To learn client-server communication and API integration.
- To understand Git and GitHub for version control.
- To gain experience in software testing and project documentation.
- To improve technical communication and professional skills.

2.5 REFERENCE

The following resources were referred to during the development of the project and preparation of this report:

- HTML Documentation – Web page structure and markup.
- CSS Documentation – Styling and responsive web design.
- JavaScript Documentation – Client-side programming and web functionality.
- Backend Framework Documentation – Server-side application development and API implementation.
- Database Documentation – Database design, queries and data management.
- Git Documentation – Git version control system documentation.
- GitHub Documentation – GitHub repository and version control documentation.

- upSkill Campus – Internship and Training Resources.

2.6 GLOSSARY

Term	Acronym
HyperText Markup Language	HTML
Cascading Style Sheets	CSS
JavaScript	JS
Application Programming Interface	API
Structured Query Language	SQL
Database Management System	DBMS
User Interface	UI
User Experience	UX
Version Control System	VCS
Bank Management System	BMS
Full Stack Web Development	FSWD
GitHub	—

3. PROBLEM STATEMENT

Banking systems are an important part of modern financial services, but real-world banking applications are often complex and require databases, secure authentication systems, network infrastructure, backend services and advanced security mechanisms.

For learning and demonstration purposes, there is a need for a simple web-based banking application that can demonstrate the fundamental operations of a banking system through a user-friendly interface.

The objective of this project is to develop a web-based Bank Management System that allows users to create an account, log in securely and perform basic banking operations through a web interface.

The system allows authenticated users to check their account balance, deposit money, withdraw money, transfer money to another registered user, view their profile and access their transaction history.

The system maintains user information, account details and transaction records using a database. Whenever a banking transaction is performed, the relevant account balance and transaction records are updated accordingly.

The project therefore aims to provide a simple, functional and user-friendly banking system while demonstrating important full-stack web development concepts such as frontend development, backend development, database management, authentication, API communication, transaction processing, validation and error handling.

4. EXISTING AND PROPOSED SOLUTION

4.1 EXISTING SOLUTION

Traditional banking applications provide a wide range of services including account management, balance checking, deposits, withdrawals, fund transfers and transaction statements.

However, real-world banking systems generally require complex databases, secure authentication mechanisms, network connectivity, backend services and large-scale infrastructure. Such systems can be difficult to understand and implement for beginners who are learning full-stack web development.

For educational purposes, a simplified web-based banking system is useful for understanding how basic banking operations, user authentication, database management and transaction processing work.

4.2 PROPOSED SOLUTION

The proposed Bank Management System is a full-stack web application developed to simulate basic banking operations.

The system provides a user-friendly web interface through which authenticated users can perform different banking operations. The frontend provides the user interface, while the backend processes requests and communicates with the database.

The proposed system provides the following functionalities:

- User account creation and registration.
- User login and authentication.
- Checking account balance.
- Depositing money.
- Withdrawing money.
- Transferring money to another registered user.
- Viewing user profile information.
- Viewing transaction history.
- Logging out of the system.
- Updating account information after banking operations.
- Validating user inputs.
- Handling errors and invalid operations.

The system communicates between the frontend and backend using web requests/APIs, while the database stores user, account and transaction information.

4.3 VALUE ADDITION

The Bank Management System provides a simple and lightweight implementation of banking operations while demonstrating important full-stack web development and software engineering concepts.

The project provides practical implementation of:

- HTML-based webpage development.
- CSS styling and responsive design.
- JavaScript programming.
- Frontend user interface development.
- Backend development.
- API creation and integration.
- Database connectivity.
- User authentication.
- Account management.
- Transaction processing.
- Data validation and error handling.
- CRUD operations.
- Testing and debugging.
- Git and GitHub based version control.

The system can also serve as a foundation for developing a more advanced banking application with stronger authentication, improved security mechanisms, advanced database architecture and additional banking features.

4.4 CODE SUBMISSION

GitHub Repository:

[https://github.com/\[YOUR-USERNAME\]/\[YOUR-REPOSITORY\]](https://github.com/[YOUR-USERNAME]/[YOUR-REPOSITORY])

The complete source code of the Bank Management System is available in the GitHub repository. The repository contains the frontend, backend, database-related files and other supporting project files required to run the web application.

4.5 REPORT SUBMISSION

The final internship report will be submitted in PDF format along with the project repository.

GitHub Repository:

https://github.com/sushmita2696_droid/upskillcampus

The final report PDF will be uploaded to the repository before the submission of the internship assessment.

5 . PROPOSED DESIGN / MODEL

The Bank Management System follows a simple full-stack web application architecture. The system allows an authenticated user to perform different banking operations through a web-based interface.

The application consists of a frontend layer, backend layer and database layer. The frontend provides the user interface and collects user input. The backend processes requests, applies business logic and communicates with the database. The database stores user information, account details and transaction records.

The overall working flow of the system is:

Start → Registration/Login → Authentication → Dashboard → Select Operation → Send Request → Backend Processing → Database Update → Display Result → Dashboard → Logout → End

The major components of the system are user authentication, user account management, banking operations, backend/API processing, database management and transaction record management.

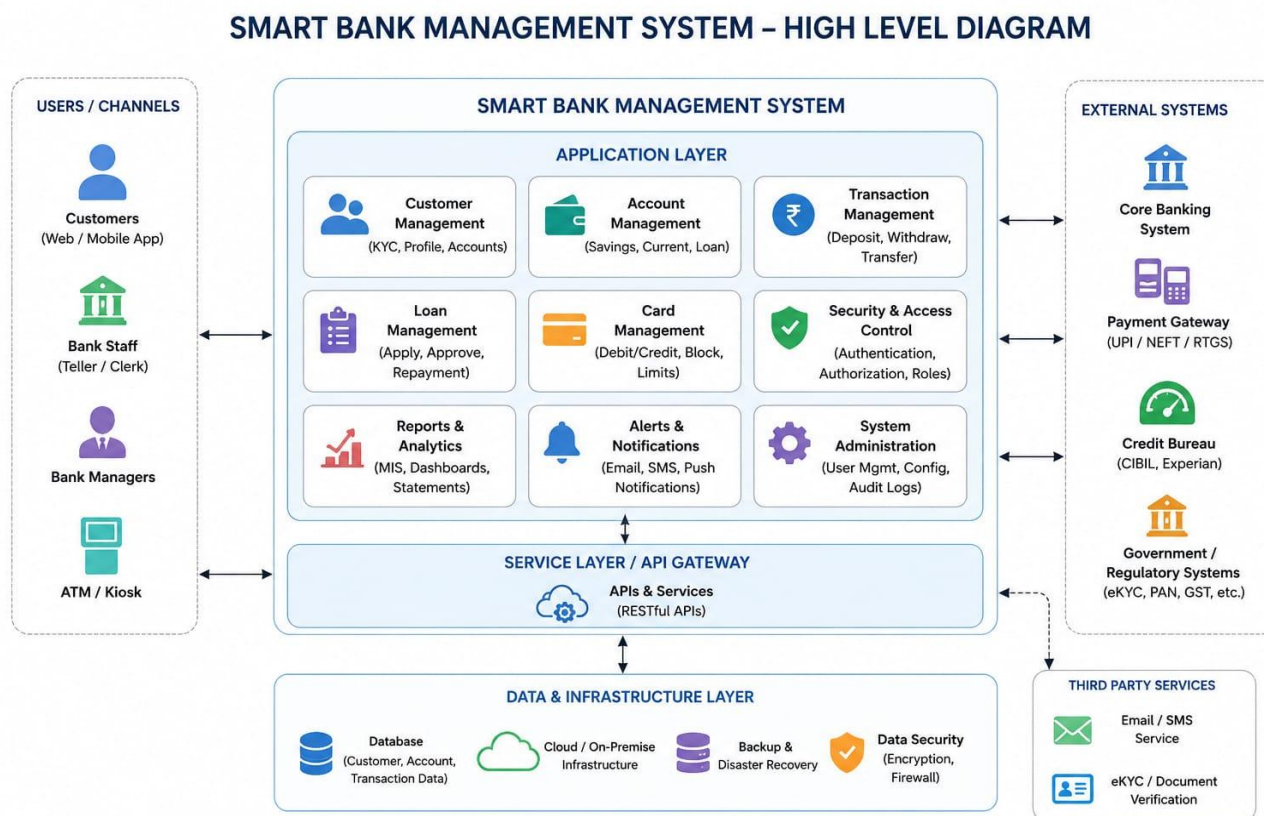
5.1 HIGH LEVEL DIAGRAM

The high-level design represents the overall flow and major components of the Bank Management System.

User → Registration / Login → Authentication → Bank Dashboard → Banking Operations → Backend / API → Database → Process Request → Display Result

Banking Operations include Create Account, Check Balance, Deposit Money, Withdraw Money, Transfer Money, View Profile, Transaction History and Logout.

Figure 1: High Level Diagram of the Bank Management System



5.2 LOW LEVEL DIAGRAM

The low-level design represents the detailed flow of user authentication and banking operations within the system.

The user first accesses the web application and chooses either registration or login. During login, the entered credentials are validated by the backend.

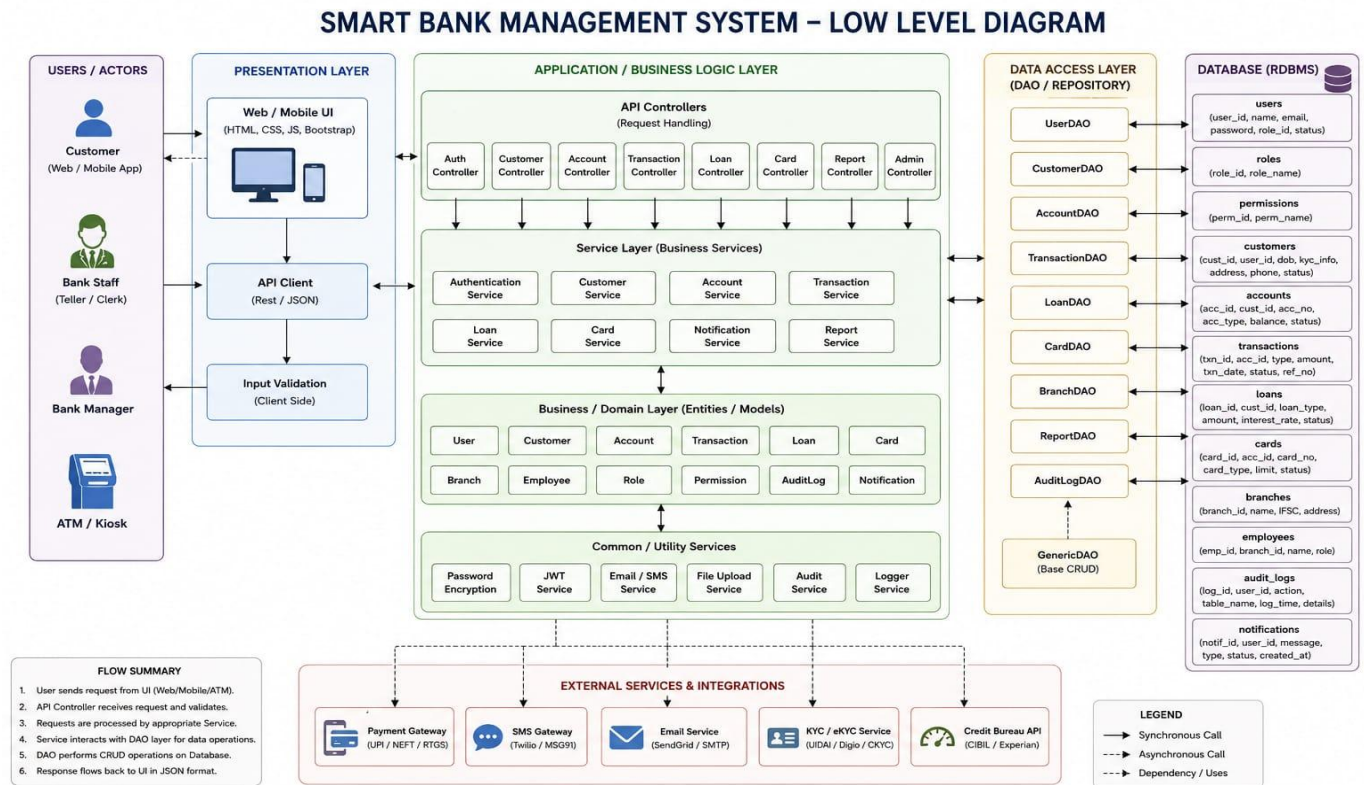
If the credentials are valid, the user is redirected to the bank dashboard. If the credentials are invalid, an appropriate error message is displayed.

From the dashboard, the user can select different banking operations such as checking balance, depositing money, withdrawing money, transferring money, viewing profile information and viewing transaction history.

The selected request is sent to the backend through an API. The backend validates the request, performs the required operation and updates the database.

The result is then returned to the frontend and displayed to the user.

Detailed Flow: Start → Open Bank Management System → Login / Sign Up → Enter User Details → Validate Credentials → Dashboard → Select Banking Operation → Send API Request → Backend Processing → Validate Data → Update Database → Return Response → Display Result → Logout → End.



5.3 USER INTERFACE

The Bank Management System provides a web-based interface for performing banking operations.

• Login Interface

The login interface allows registered users to enter their credentials and access their bank account.

• Registration Interface

The registration interface allows a new user to create a bank account by entering the required personal and account information.

• Dashboard Interface

The dashboard provides access to different banking operations after successful authentication.

• Deposit Money Interface

The user can enter an amount and deposit money into their account.

- **Withdraw Money Interface**

The user can enter an amount and withdraw money when sufficient balance is available.

- **Transfer Money Interface**

The user can enter the receiver's account details and transfer money.

- **User Profile Interface**

The profile interface displays information related to the authenticated user.

- **Transaction History Interface**

The transaction history page displays previous deposits, withdrawals and transfers.

- **Logout Interface**

The logout functionality securely ends the user's current session.

6. PERFORMANCE TEST

The Bank Management System was tested to verify the correctness, reliability and functionality of its major banking operations. Testing was performed by executing different operations through the web interface and comparing the expected results with the actual results.

The testing process focused on user registration, user authentication, account balance management, deposits, withdrawals, money transfers, transaction history, database operations, API communication and input validation.

6.1 TEST PLAN / TEST CASES

Test Case 1: User Registration

Expected Result: A new user account should be created successfully.

Actual Result: The user account was created successfully.

```

=====
                        CREATE ACCOUNT
=====

Enter account holder name: Sushmita Poddar
Enter phone number: 9876543210
Enter email address: sushmita@gmail.com
Enter address: Kolkata

Select Account Type:
1. Savings Account
2. Current Account
Enter choice: 1
Enter initial deposit: 5000

=====
                        ACCOUNT CREATED SUCCESSFULLY
=====

Account Number : 1001
Account Holder : Sushmita Poddar
Account Type   : Savings
Initial Balance: ₹5000.00
=====

```

Status: Passed

Test Case 2: Check account balance

Expected Result: The current account balance should be displayed correctly.

Actual Result: The correct account balance was displayed.

```

=====
                        BALANCE ENQUIRY
=====

Enter account number: 1001

Account Holder : Sushmita Poddar
Account Number : 1001
Current Balance: ₹5000.00

```

Status: Passed

Test Case 3: Deposit money

Expected Result: The deposited amount should be added to the account balance.

Actual Result: The account balance was increased correctly.

```
=====
                        DEPOSIT MONEY
=====

Enter account number: 1001
Enter amount to deposit: 2000

Deposit successful!
Deposited Amount : ₹2000.00
New Balance      : ₹7000.00
```

Status: Passed

Test Case 4: Withdraw money

Expected Result: The withdrawn amount should be deducted from the account balance when sufficient balance is available.

Actual Result: The account balance was decreased correctly.

```
=====
                        WITHDRAW MONEY
=====

Enter account number: 1001
Enter amount to withdraw: 2000

Withdrawal successful!
Withdrawn Amount : ₹2000.00
Remaining Balance: ₹5000.00
```

Status: Passed

Test Case 5: Transfer money

Expected Result: The specified amount should be deducted from the sender's account and added to the receiver's account.

Actual Result: The transfer was completed successfully and both account records were updated.

```
=====
                        MONEY TRANSFER
=====
Enter sender account number: 1001
Enter receiver account number: 1002
Enter transfer amount: 1000

=====
                        TRANSFER SUCCESSFUL
=====
From Account : 1001
To Account   : 1002
Amount       : ₹1000.00
Sender Balance: ₹4000.00
Receiver Balance: ₹6000.00
```

Status: Passed

Test Case 6: View transaction history

Expected Result: The system should display previous banking transactions.

Actual Result: Transaction history was displayed correctly.

```
=====
                        TRANSACTION HISTORY
=====
Enter account number: 1001

Account Holder: Sushmita Poddar
Account Number: 1001

Transaction History:
-----
Initial Deposit: ₹5000.00
Deposit: +₹2000.00 | Balance: ₹7000.00
Transfer Received: +₹2000.00 | Balance: ₹5000.00
-----
Current Balance: ₹5000.00
```

Status: Passed

Test Case 7: View user profile

Expected Result: The authenticated user's profile information should be displayed.

Actual Result: The user profile information was displayed correctly.

```
-----  
Account Number : 1001  
Account Holder : Sushmita Poddar  
Phone Number   : 9876543210  
Email          : sushmita@gmail.com  
Address        : Kolkata  
Account Type   : Savings  
Balance        : ₹5000.00  
-----
```

Status: Passed

Test Case 8: Logout

Expected Result: The user should be logged out and redirected to the login page.

Actual Result: The user was successfully logged out.

```
-----  
BANKING MENU  
-----  
1. Create Account  
2. Display All Accounts  
3. Deposit Money  
4. Withdraw Money  
5. Check Balance  
6. Transfer Money  
7. Transaction History  
8. Search Account  
9. Delete Account  
10. Exit  
-----  
Enter your choice: 10  
  
Thank you for using Banking Information System!  
Have a nice day!
```

Status: Passed